

COMP 3311 数据库管理系统 — 课程总结 (Part 1: Lectures 1–7)

课程: COMP 3311 Database Management Systems
时间: 2026年6月15日–19日
覆盖: Lecture 1 – Lecture 7（期中考试前内容）

目录

- 1. Lecture 1: 数据库管理系统概述
- 2. Lecture 2: E-R 数据模型（上）
- 3. Lecture 3: E-R 数据模型（下） — 约束与设计选择
- 4. Lecture 4: 关系数据库设计 — E-R 到关系模式的规约
- 5. Lecture 5: 关系数据库设计 — 函数依赖
- 6. Lecture 6: 关系数据库设计 — 范式化
- 7. Lecture 7: 关系代数

1. Lecture 1: 数据库管理系统概述

1.1 核心概念

术语	定义
数据库 (Database)	有组织地收集的大量相关数据，具有特定目的、反应真实世界、包含语义信息
DBMS	通用软件系统，用于定义、存储、操作、共享和保护数据库
数据模型 (Data Model)	描述数据组织和访问方式的语言，包含：数据结构类型、完整性约束、操作
模式 (Schema)	用数据模型描述的数据库结构（相对稳定）
实例 (Instance)	数据库中某个时间点的实际数据内容（频繁变化）

1.2 为什么需要 DBMS？（文件系统的 7 大缺陷）

问题	说明
数据重复与不一致	同一数据存于多处，容易不一致
难以满足未预见需求	数据结构专为特定应用定制
相关数据隔离	关联数据散落在不同文件和格式中
约束管理困难	约束嵌入程序代码中，难以修改和执行

问题	说明
更新缺乏原子性	系统故障可能导致数据不一致（如转账）
并发访问问题	多用户同时访问导致数据丢失（如 lost update）
安全性难以保障	临时性数据管理导致安全策略难以实施

1.3 DBMS 的核心原则

- **集成统一**组织数据
- 将**元数据**（系统目录）与**原始数据**分离
- 支持**多用户**
- **集中控制**数据定义和访问

1.4 常见数据模型对比

模型	数据组织	关系表示	是否使用模式
E-R	实体与关系	显式（图中表示）	是
关系模型	表（Table）	隐式（数据中）	是
Key-Value	键-值对	聚合在数据中	否
文档模型	文档（如JSON）	聚合在数据中	否

1.5 三级数据抽象（仅关系型 DBMS）

View Level（视图层）– 为特定应用提供数据子集，可隐藏/添加信息

↑ Logical Data Independence

Logical Level（逻辑层）– 描述存什么数据及其关系

↑ Physical Data Independence

Physical Level（物理层）– 描述数据如何存储在磁盘上

- **逻辑数据独立性**：修改逻辑模式不影响视图模式
- **物理数据独立性**：修改物理模式不影响逻辑模式

1.6 DBMS 总体架构

用户：Naïve / Application Programmers / Sophisticated / DBA

Query Processor:
DDL 解释器 + DML 编译器 + 查询评估引擎
Storage Manager:
文件管理器 + 缓冲管理器 + 授权完整性管理器 + 事务管理器
磁盘存储: 数据 + 索引 + 系统目录 + 统计数据

- **DDL** (数据定义语言): 定义数据库模式
- **DML** (数据操作语言): 访问和操作数据
 - **过程式 DML**: 指定需要什么数据以及如何获取 (如 MQL)
 - **声明式 DML**: 只指定需要什么数据 (如 SQL)

1.7 用户类型

用户类型	描述
Naïve Users	使用现有应用程序 (如打印报表)
Application Programmers	使用 DML 开发访问 DBMS 的应用程序
Sophisticated Users	直接使用查询语言或分析工具
DBA	协调 DBMS 的所有活动 (模式、物理组织、性能、权限)

2. Lecture 2: E-R 数据模型 (上)

2.1 数据库设计流程

需求分析 → 概念模式(ER图) → 逻辑模式(DDL) → 物理模式 → 实施

- **需求分析**: 理解应用领域, 识别数据需求
- **逻辑数据库设计**: 概念模式 (E-R 图, DBMS 无关) → 逻辑模式 (DDL, DBMS 相关)
- **物理数据库设计**: 描述数据如何存储在存储介质上

2.2 E-R 模型三大基本概念

实体 (Entity)

- 应用中我们想存储数据的**事物** (如 Employee, Student, Course)

- 实体类型：对所有实体实例的公共描述
- 实体集：所有实体实例的集合
- 每个实体实例**具有唯一标识** (identity)

属性 (Attribute)

- 实体的**属性**，描述该属性的数据值
- 属性类型：
 - **简单/单值属性** — 基本属性
 - **复合属性** — 可分解的 (如 address → streetNo, streetName)
 - **多值属性** — 用 { } 标记 (如 {skill})
 - **派生属性** — 可从其他属性计算得出，用 () 标记 (如 (age) 从 birthdate 派生)
- 属性值可以为 **null** (缺失、未知或不适用)

关系 (Relationship)

- 实体之间的**关联描述**
- 关系本质上是**双向的**
- 关系可以有自己属性 (如 WorksOn 中的 %time)
- 关系度 (Degree)：
 - **一元 (unary)**：同一实体与自己关联
 - **二元 (binary)**：两个不同实体关联 (最常见)
 - **三元 (ternary)**：三个实体关联 (极少见)
- **角色名 (Role Name)**：用于一元关系，标识实体在关系中扮演的角色

2.3 泛化/特化 (Generalization/Specialization)

- **超类型 (Supertype)** → 共性属性
- **子类型 (Subtype)** → 特有属性
- 分类方式：
 - **用户定义 (User-defined)**：由设计者指定子类型归属
 - **谓词定义 (Predicate-defined)**：由鉴别器属性 (discriminator) 值决定

2.4 继承 (Inheritance)

- 子类型从超类型**继承**所有属性和关系
 - 子类型可**添加**新的属性/关系
 - 设计指南：继承不应超过 2-3 层
 - **多重继承**：子类型从多个超类型继承，可能导致命名冲突 (通过重命名解决)
-

3. Lecture 3: E-R 数据模型（下） — 约束与设计选择

3.1 约束概述

约束是对数据的**逻辑限制**，任何数据都必须满足。约束为数据添加额外的语义。

重要认识：不是所有约束都能/应该在 E-R 模型中表示——有些留给应用程序处理更合适。

3.2 属性约束

域约束 (Domain Constraint)

- 限制属性的取值类型和范围
- 可通过**数据类型**（如 integer, char(20)）或**谓词**（如 salary 在 0–100,000）来指定

键约束 (Key Constraint)

- **候选键 (Candidate Key)**：最小属性集，能唯一标识一个实体实例
- **主键 (Primary Key)**：从候选键中选出的一个，DBMS 自动强制唯一性
- **复合键 (Composite Key)**：由多个属性组成
- 其他候选键的唯一性**不会被** DBMS 自动强制

3.3 强实体 vs 弱实体

特性	强实体 (Strong Entity)	弱实体 (Weak Entity)
主键	有	无
存在依赖	独立存在	依赖标识实体
标识方式	自有主键	标识实体主键 + 鉴别器（如有）
E-R 图中关系线	虚线	实线（标识关系）

3.4 泛化约束：覆盖 (Coverage)

不相交性 (Disjointness)

- **不相交 (disjoint)**：超类型实例最多属于一个子类型
- **重叠 (overlapping)**：超类型实例可属于多个子类型

完备性 (Completeness)

- **完全 (total)**: 超类型实例**必须**属于至少一个子类型
- **部分 (partial)**: 超类型实例**不必须**属于任何子类型

常见组合: disjoint,total | disjoint,partial | overlapping,total | overlapping,partial

3.5 关系约束: 基数与参与

基数约束 (Cardinality) = 最大参与数:

- `max-card = 1`: 一对一 (1:1) 或一对多 (1:N)
- `max-card = N`: 多对多 (N:M)

参与约束 (Participation) = 最小参与数:

- `min-card = 0`: **部分参与** (partial) — 不必须
- `min-card ≥ 1`: **完全参与** (total) — 必须

```

Department  ——^—— WorksIn  ——|—— Employee
                (1,N)          (0,1)
                total参与      partial参与
  
```

3.6 排斥约束 (Exclusion/XOR)

实体实例在多个关系中**最多只能参与一个** (不能同时属于两个)。例如: Task 要么属于 InternalProject **要么**属于 ExternalProject, 不能两者都是。

3.7 设计选择

对比	何时选择
实体 vs 属性	当概念有多个属性/关系时用实体; 结构简单时用属性
强实体 vs 弱实体	有唯一标识时用强实体; 没有唯一标识时用弱实体
实体 vs 关系	当概念代表应用域中独立的事物时用实体; 当两个实体之间的关联仅为单一事实时用关系
属性放在哪	属性依赖一个实体 → 放该实体; 属性仅当关系存在时才存在 → 放关系上

关键限制: 在同一个关系类型中, 两个实体实例之间最多只能有一条关系实例 → 这就是为什么一个人在同一支行开两个账户时需要把 Account 做成实体。

3.8 Crow Foot 符号汇总

概念	符号
实体	矩形框
属性	列在实体框内
主键	下划线
多值属性	{ }
派生属性	()
1:1 关系	`——
1:N 关系	`——  ——
N:M 关系	 ——  ——
标识关系（弱实体）	实线 
排斥约束	X 符号 + 多关系
泛化/特化	三角+圆圈符号

4. Lecture 4: 关系数据库设计 — E-R 到关系模式的规约

4.1 关系数据模型基础

概念	说明
关系 (Relation)	表，记为 $R(A_1, A_2, \dots, A_n)$
属性 (Attribute)	列
域 (Domain)	属性的类型和取值范围
元组 (Tuple)	行
度 (Degree)	属性的数量
势 (Cardinality)	元组的数量

关系的性质：

- 元组是**无序的**（关系是集合）
- 所有属性值必须是**原子的**（不允许复合/多值属性）
- 关系实例是域的笛卡尔积的**任意子集**： $r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$

4.2 键 (Keys)

Superkey (超键) → 任何能唯一标识元组的属性集



Candidate Key (候选键) → 最小的超键 (不能再去掉任何属性)



Primary Key (主键) → 设计者选出的一个候选键

- **实体完整性约束**: 主键不能包含 null 值
- **参照完整性约束** (外键 Foreign Key): T.fk_s 的值要么等于 S 中某元组的主键值, 要么全部为 null
- 外键表示 1:1 或 1:N 关系

4.3 E-R 模式到关系模式的规约规则

强实体

$E(k_s, \dots) \rightarrow R_s(k_s, \dots)$

弱实体

T (弱实体, 鉴别器 d_t) 依赖 S (强实体, 主键 k_s)
→ R_t(fk_s, d_t, a_r, ...)
PK = (fk_s, d_t)
FK: fk_s REFERENCES S(k_s) ON DELETE CASCADE

复合属性

- 选项 1: 拼接为一个属性
- 选项 2: 拆分为多个独立属性

多值属性

$S(k_s, \dots, \{m\})$
→ $S(k_s, \dots) + SM(fk_s, m)$ (PK = 全部属性)
FK: fk_s REFERENCES S(k_s) ON DELETE CASCADE

泛化/特化

- **选项 1** (通用): 每个实体一个关系模式。子类型添加父类型主键作为外键+主键, ON DELETE CASCADE。

- **选项 2** (仅 total, disjoint 可用): 只创建子类型关系模式, 包含父类型全部属性。

二元关系

$R(a_r, \dots)$ between $S(k_s, \dots)$ and $T(k_t, \dots)$

$1:1 \rightarrow R_r(fk_s, fk_t, a_r)$ PK = fk_s 或 fk_t

$1:N \rightarrow R_r(fk_s, fk_t, a_r)$ PK = fk_t (N 侧)

$N:M \rightarrow R_r(fk_s, fk_t, a_r)$ PK = (fk_s, fk_t)

模式合并 (优化)

- 1:1: 关系模式可与任一方实体合并
- 1:N: 关系模式可与 N 侧实体合并
- 合并后外键的参照完整性动作由参与约束决定:
 - partial $\rightarrow$ ON DELETE SET NULL
 - total $\rightarrow$ ON DELETE CASCADE

排斥约束

- 选项 1: 单一关系, 两个外键 + CHECK 约束确保只有一个非 null
- 选项 2: 单一关系, 合并外键 + 类型判别属性 (仅当键域相同时可用)

4.4 代理键 (Surrogate Key)

- 新增的无应用意义的属性, 用作主键
- 用途: 将弱实体变为强实体, 或替代过长的复合主键
- **绝不引入到 E-R 模式中**, 只在规约到关系模式时添加

5. Lecture 5: 关系数据库设计 — 函数依赖

5.1 函数依赖 (FD) 定义

在关系模式 R 中, $X \rightarrow Y$ 意味着: **任何时候**, 给定 X 的值, 最多只有一个 Y 的值与之对应。

- **平凡 FD (Trivial FD)**: $Y \subseteq X$ (永远成立)
- **非平凡 FD (Nontrivial FD)**: $Y \cap X = \emptyset$ (作为约束, 约束合法的关系实例)

5.2 Armstrong 公理系统

规则	名称	含义
IR1	自反律 (Reflexivity)	如果 $Y \subseteq X$, 则 $X \rightarrow Y$
IR2	增广律 (Augmentation)	$X \rightarrow Y \models XZ \rightarrow YZ$
IR3	传递律 (Transitivity)	$X \rightarrow Y, Y \rightarrow Z \models X \rightarrow Z$

附加规则 (可由 IR1-IR3 推导):

| IR4 | 合并律 (Union) | $X \rightarrow Y, X \rightarrow Z \models X \rightarrow YZ$ |

| IR5 | 分解律 (Decomposition) | $X \rightarrow YZ \models X \rightarrow Y$ 和 $X \rightarrow Z$ |

| IR6 | 伪传递律 (Pseudo-transitivity) | $X \rightarrow Y, WY \rightarrow Z \models WX \rightarrow Z$ |

- Armstrong 公理是**可靠 (sound)** 且**完备 (complete)** 的
- **F 的闭包 F^+** : 从 F 推出的所有 FD 的集合

5.3 属性闭包 (Attribute Closure)

X^+ (X 在 F 下的闭包) 是由 X 在函数依赖 F 下能确定的所有属性的集合。

$X \rightarrow Y$ 属于 F^+ **当且仅当** $Y \subseteq X^+$

计算算法:

```
 $X^{(0)} = X$ 
Repeat:
     $X^{(i+1)} = X^{(i)} \cup Z$ 
    (where  $\exists Y \rightarrow Z$  in F and  $Y \subseteq X^{(i)}$ )
Until  $X^{(i+1)} = X^{(i)}$ 
Return  $X^{(i+1)}$ 
```

属性闭包的用途:

1. 测试超键: X^+ 包含 R 的所有属性 $\rightarrow X$ 是超键; 如果 X 最小 $\rightarrow X$ 是候选键
2. 测试 FD: 要检验 $X \rightarrow Y$ 是否在 F^+ 中, 计算 X^+ 并检查 $Y \subseteq X^+$
3. 计算 F^+ : 对每个 $X \subseteq R$, 计算 X^+ , 输出 $X \rightarrow Y$ (对所有 $Y \subseteq X^+$)

素属性 (Prime Attribute): 属于某个候选键的属性; 否则为非素属性。

5.4 规范覆盖 (Canonical Cover) F_c

- F_c 与 F 等价 (拥有相同的 F^+)
- F_c 无冗余属性
- F_c 中每个 FD 的 LHS 唯一

计算算法:

1. $F_c = F$
2. 重复:
 - 使用合并律 (IR4) 合并 LHS 相同的 FD
 - 找到并删除无关属性 (extraneous attributes)
3. 直到 F_c 不再变化

示例: $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\} \rightarrow F_c = \{A \rightarrow B, B \rightarrow C\}$

6. Lecture 6: 关系数据库设计 — 范式化

6.1 范式化目标

范式化利用函数依赖将**不满足要求**的关系模式**分解**为更理想的关系模式。

设计五大指南:

指南	内容
Guideline 1	清晰的属性语义 — 不要把多个实体类型混在一个关系模式中
Guideline 2	最小化 null 值 — 避免将有大量 null 的属性放在同一模式中
Guideline 3	最小化冗余 (消除操作异常) — 插入/删除/更新异常
Guideline 4	无损分解 (Lossless Join) — 连接后能恢复原关系
Guideline 5	保持函数依赖 — FD 不要分散在多个关系模式中 (避免 join 才能验证约束)

无损分解条件 ($R \rightarrow R_1, R_2$): $R_1 \cap R_2 \rightarrow R_1$ 或 $R_1 \cap R_2 \rightarrow R_2$ 在 F^+ 中。
(公共属性必须是 R_1 或 R_2 的超键。)

6.2 各级范式总结

1NF $\supset$ 2NF $\supset$ 3NF $\supset$ BCNF

(越往后要求越严格)

第一范式 (1NF)

- 所有属性值都是**原子**的 (无复合、无多值)
- 关系模型的固有属性, 规约出的关系模式永远满足 1NF

第二范式 (2NF)

所有**非素属性**对**每个候选键**都是**完全函数依赖**的。

- 消除**部分依赖**: 候选键的真子集 $\rightarrow$ 非素属性

示例 (违反 2NF):

Car(make, model, engineSize, fee, origin, tax)

PK: (make, model, engineSize)

违反 FD: engineSize $\rightarrow$ fee $\leftarrow$ engineSize 是 PK 的真子集, fee 是非素属性

修复: 分解为 Licensing(engineSize, fee) 和 Car(make, model, engineSize, origin, tax)。

第三范式 (3NF)

在 2NF 基础上, 所有**非素属性**对每个候选键都是**非传递依赖**的。

即: 对于任意 $X \rightarrow A$ 在 F^+ 中, 至少满足其一:

1. $A \in X$ (平凡 FD)
2. X 是超键 (superkey)
3. A 是素属性 (prime attribute)

- 消除**传递依赖**: 非超键 $\rightarrow$ 非素属性

示例 (违反 3NF):

Car(make, model, engineSize, origin, tax)

违反 FD: $\text{origin} \rightarrow \text{tax} \leftarrow \text{origin}$ 不是超键, tax 是非素属性

3NF 分解 (合成) 算法

计算 F_c (规范覆盖)

$S = \emptyset$

对每个 FD $X \rightarrow Y$ 在 F_c 中:

$S = S \cup (X, Y)$

如果 S 中没有任何模式包含 R 的候选键:

$S = S \cup K$ (添加一个候选键)

- **总是**产生无损连接、依赖保持的 3NF 分解。

Boyce-Codd 范式 (BCNF)

每个 FD 的**行列式 (LHS)** 都是超键。

即: 对于任意 $X \rightarrow A$ 在 F^+ 中, 至少满足其一:

1. $A \in X$ (平凡 FD)
2. X 是超键

- 比 3NF 更严格: 3NF 允许"A 是素属性"的例外, BCNF 不允许
- BCNF = **无冗余** (可由 FD 消除的冗余)
- BCNF **不一定是**依赖保持的!

示例:

$\text{Car}(\text{make}, \text{model}, \text{engineSize}, \text{origin})$

违反 FD: $\text{origin} \rightarrow \text{engineSize} \leftarrow \text{origin}$ 不是超键

分解 $\rightarrow \text{Car}(\text{make}, \text{model}, \text{origin}) + \text{Country}(\text{origin}, \text{engineSize})$

但这丢失了 FD: $\text{make}, \text{model}, \text{engineSize} \rightarrow \text{origin}$ **×** 不保持依赖!

BCNF 分解算法

$S = \{R\}$

直到 S 中所有模式都在 BCNF 中:

对违反 BCNF 的 $X \rightarrow Y$:

$S = (S - R) \cup (R - Y) \cup (X, Y)$

- 总是产生**无损连接**的分解
- 但**不能保证依赖保持**

6.3 范式选择指南

范式	优点	缺点
3NF	总是存在依赖保持的分解	可能包含一些冗余
BCNF	消除更多冗余	可能不存在依赖保持的分解

实用指导：实践中，3NF 对大多数应用来说"够好了"。

7. Lecture 7: 关系代数

7.1 关系查询语言

类型	描述	代表
关系代数 (RA)	过程式 — 描述 如何 计算	基本操作 + 附加操作
关系演算 (RC)	声明式 — 只描述 要什么	如 SQL 的基础

闭包性质：关系代数的每次操作输入一个/多个关系，输出也是关系 → 操作可以**组合**（嵌套）。

7.2 基本操作

操作	符号	说明
选择 (Selection)	$\sigma_C(R)$	选择满足条件 C 的元组（行）；模式不变
投影 (Projection)	$\pi_L(R)$	保留列表 L 中的属性（列）； 自动去重
并 (Union)	$R_1 \cup R_2$	属于 R_1 或 R_2 的元组；要求 并兼容 （同属性数+同类型）
差 (Set Difference)	$R_1 - R_2$	属于 R_1 但不属于 R_2 的元组
笛卡尔积 (Cartesian Product)	$R_1 \times R_2$	将 R_1 每个元组与 R_2 每个元组组合（巨大结果）

7.3 附加操作

操作	符号	说明
交 (Intersection)	$R_1 \cap R_2$	同时属于两者的元组；可由差表示
θ -连接 (θ -Join)	$R_1 \bowtie_C R_2$	笛卡尔积 + 选择 = $\sigma_C(R_1 \times R_2)$
等值连接 (Equijoin)	—	连接条件全为等值
自然连接 (Natural Join)	$R_1 \bowtie R_2$	基于同名属性等值的等值连接，结果只保留一个共同属性副本
左外连接 (Left Outer Join)	$R_1 \bowtie\!\!\!\bowtie R_2$	自然连接 + R_1 中不匹配的元组（填充 null）
右外连接 (Right Outer Join)	$R_1 \bowtie\!\!\!\bowtie R_2$	自然连接 + R_2 中不匹配的元组（填充 null）
全外连接 (Full Outer Join)	$R_1 \bowtie\!\!\!\bowtie R_2$	自然连接 + 双方不匹配的元组（填充 null）
更名 (Rename)	$\rho_x(E)$	给表达式结果重命名

7.4 操作总结

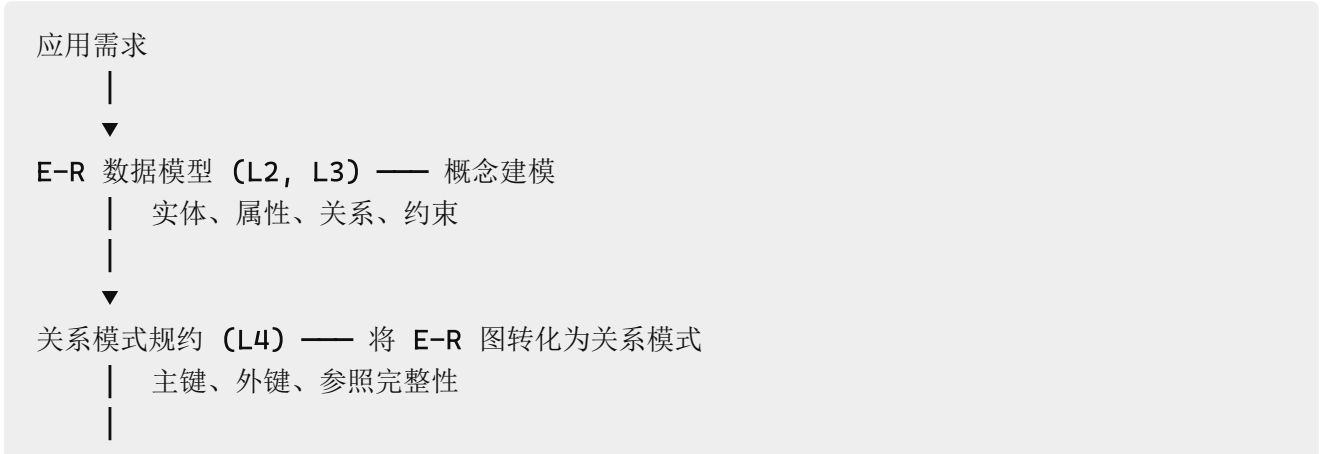
基础操作（不能由其他操作表示）：

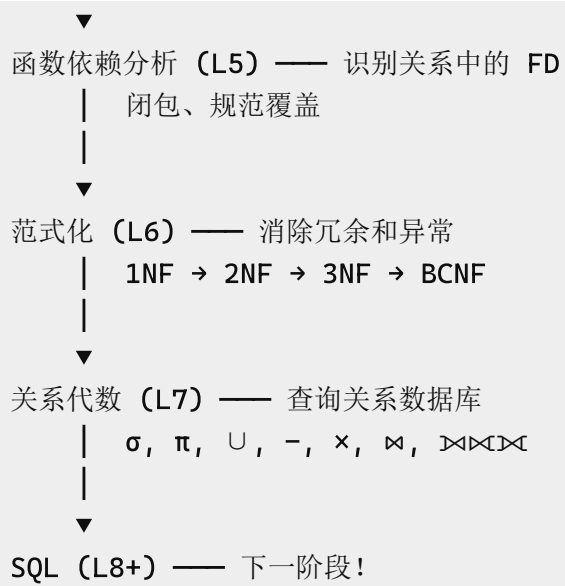
- σ 选择（一元）
- π 投影（一元）
- $\cup$ 并（二元, union-compatible）
- $-$ 差（二元, union-compatible）
- $\times$ 笛卡尔积（二元）

附加操作（可由基础操作表示）：

- $\cap$ 交 = $((R \cup S) - (R - S)) - (S - R)$
- $\bowtie$ 连接 = $\sigma_C(R \times S)$
- $\bowtie\!\!\!\bowtie$ 外连接 = 自然连接 + null填充
- ρ 更名
- $\leftarrow$ 赋值

核心概念关联图





课程参考: "Fundamentals of Database Systems" (教材章节标注于各 Lecture 幻灯中)